

CoAdapt-GUI: Joint Workflow Context and Policy Adaptation for Unseen GUI Applications

Linqiang Guo^{1,2} Li Gu^{1,2} Zihuan Jiang³ Zhixiang Chi³ Siobhan Reid^{1,2}
 Ziqiang Wang^{1,2} Yuanhao Yu⁴ Wei Liu¹ Yang Wang^{1,2} Tse-Hsun (Peter) Chen¹

¹ Concordia University ² Mila – Québec AI Institute ³ University of Toronto ⁴ McMaster University

Abstract

Mobile GUI agents remain brittle when deployed to applications absent from source training. We study novel-app generalization under a limited target interaction budget and without target demonstrations. We introduce CoAdapt-GUI, a test-time adaptation (TTA) framework that jointly adapts structured workflow context and policy from the agent’s own target-app rollouts and rewards. The workflow context retains transferable procedures, failure modes, and verification rules while excluding app-bound source details. This separation allows reusable workflow knowledge to guide adaptation without transferring source-interface state. For policy adaptation, task–context-matched group-relative optimization updates a LoRA adapter on a frozen vision–language model. Across two unseen-app evaluations, CoAdapt-GUI reaches 45.0% on AndroidWorld-Generalization, compared with 37.5% for the reported Policy-Only TTA baseline, and raises AndroidWorld Plus performance from 38.6% to **52.9%**. These results show that transfer-constrained workflow context provides substantial gains and that joint policy adaptation further improves held-out performance.

1 Introduction

Mobile GUI agents execute natural-language instructions by interpreting application interfaces and carrying out multi-step actions. Recent advances in GUI grounding, trajectory synthesis, and reinforcement learning have substantially improved their performance on established benchmarks (Cheng et al. 2024; Hong et al. 2024; Cheng et al. 2026; Xu et al. 2025). However, these evaluations generally assume a predefined collection of applications and workflows (Rawles et al. 2025; Xu et al. 2024; Kong et al. 2026). In deployment, an agent may instead encounter an application whose interface and task procedures were absent from training. Adapting to such applications from limited interaction is therefore essential for effective operation beyond the original training environment.

This challenge is particularly pronounced under cross-application shifts. AndroidWorld-Generalization distinguishes generalization to unseen task instances, templates, and applications. Source-side online RL improves a 7B policy by 26.1 percentage points on unseen instances, but the gain drops to 8.3 points on unseen applications (Gu et al. 2026). Success on new tasks within a familiar interface there-

fore does not imply effective behavior in an unfamiliar application.

An unseen application can expose both interface-specific and procedural gaps. An agent may understand the goal but fail to ground actions to unfamiliar interface elements, or it may execute individual actions correctly yet lack the workflow and completion conditions needed to finish the task. Updating the policy can change how the agent interprets and acts on the new interface, while adapting an external workflow context can preserve procedures learned from target-side experience. Updating only one state may therefore leave an important source of failure unresolved.

Existing work only partially addresses this target-side adaptation problem. AndroidWorld-Generalization updates the policy from the agent’s own target-app rollouts while leaving its workflow context fixed (Gu et al. 2026). UI-Mem jointly learns experience memory and policy during source-side training, then transfers the resulting knowledge to unseen applications (Xiao et al. 2026). This leaves open whether an agent can use interactions collected after encountering a new application to adapt both its workflow context and policy before evaluation on held-out target tasks.

We introduce **CoAdapt-GUI**, a test-time adaptation (TTA) framework that maintains and jointly updates these two states from target-app rollouts and task-level rewards. Its context channel contrasts successful and failed traces to revise transferable procedures, failure patterns, and completion checks while excluding app-bound details. Its policy channel updates a lightweight LoRA adapter through a task–context-conditioned group-relative objective (Shao et al. 2024), while keeping the VLM backbone frozen. Both updates are derived from the same rollout groups within each adaptation round, and the resulting context and policy are frozen before evaluation on held-out target tasks.

We evaluate CoAdapt-GUI under two levels of unseen-app generalization. AndroidWorld-Generalization (Gu et al. 2026) evaluates new instances of task types encountered during target adaptation. In this setting, CoAdapt-GUI reaches 45.0%, exceeding the reported Policy-Only TTA baseline of 37.5% by 7.5 percentage points. To test whether adaptation transfers beyond the task types encountered during target interaction, we construct AndroidWorld Plus with disjoint adaptation and evaluation task types within each target app. Here, Context-Only TTA improves the Base Policy from

38.6% to 48.1%, while CoAdapt-GUI further raises success to **52.9%**. These results show that target-grounded workflow adaptation provides substantial gains and that jointly adapting the policy achieves the best overall performance in both settings.

This paper makes three contributions:

- **Joint target-side adaptation from autonomous interaction.** We introduce a test-time adaptation framework that updates separate workflow-context and policy states using only the agent’s own target-app rollouts and task rewards, without target demonstrations or access to held-out evaluation signals.
- **Transfer-constrained context-policy adaptation.** We separate transferable workflow knowledge from app-bound source state and coordinate two reward-guided updates from the same target interactions. Validated reward differences revise the workflow context, while policy credit is computed only among rollouts sharing the same task and context condition.
- **Evaluation across instance- and task-type generalization.** We evaluate CoAdapt-GUI on the released AndroidWorld-Generalization unseen-app split and construct AndroidWorld Plus to hold out entire task types during target adaptation. CoAdapt-GUI reaches 45.0% and 52.9% in the two settings, respectively, achieving the best overall result in both settings.

2 Related Work

2.1 Mobile GUI Agents and Generalization

Mobile GUI agents commonly combine a vision-language policy with an interaction loop for action execution and task verification. GUI-specific pretraining and high-resolution perception improve visual grounding and action prediction (Cheng et al. 2024; Hong et al. 2024; Gou et al. 2024), while OpenMobile and MobileRL scale supervised or online policy learning from executable trajectories (Cheng et al. 2026; Xu et al. 2025). UI-Mem also evaluates zero-shot transfer to held-out applications by retrieving hierarchical memory accumulated during source-side policy training (Xiao et al. 2026). These approaches acquire transferable behavior on source applications before evaluation; we instead study how an agent can continue adapting after encountering an unseen application.

AndroidWorld, AndroidLab, and MobileWorld provide programmatically evaluated environments for studying GUI agents across apps and multi-app workflows (Rawles et al. 2025; Xu et al. 2024; Kong et al. 2026). Related studies examine generalization across websites, tasks, applications, and app categories (Deng et al. 2023; Li et al. 2024). Most closely, AndroidWorld-Generalization separates unseen instances, templates, and apps, and shows that policy adaptation on target-app interactions can improve unseen-app performance (Gu et al. 2026). We use its released setting for comparison, but adapt both workflow context and the policy from autonomously collected target rollouts.

2.2 Test-Time Context and Policy Adaptation

Test-time adaptation updates a deployed model using data from its target environment (Sun et al. 2020; Wang et al. 2021). Recent methods extend this idea to language-model reasoning and interactive agents (Zuo et al. 2025; Zweiger et al. 2025; Chen et al. 2026). Few-shot GUI methods such as LearnAct and AdaptAgent also use target experience, but rely on demonstrations rather than the agent’s own reward-bearing interaction (Liu et al. 2025a; Verma et al. 2024). Our setting instead uses executable rewards from autonomously collected trajectories and evaluates adaptation on held-out target tasks.

External memory provides a complementary in-context adaptation channel. AppAgent records explored app functionality, Agent Workflow Memory abstracts reusable routines, and Mobile-Agent-E and MobiMem accumulate experience as evolving notes or prompt memory (Zhang et al. 2025; Wang et al. 2024, 2025; Liu et al. 2025b). These methods demonstrate the value of explicit workflow knowledge while generally keeping the underlying policy fixed. Unlike UI-Mem, which varies memory guidance within each policy-optimization group to internalize guided behavior (Xiao et al. 2026), CoAdapt-GUI treats context and policy as separate target-side states. Reward differences across context variants revise the workflow state, while policy credit is computed only among rollouts sharing the same task and context condition. Agent-SAMA additionally represents app execution as a finite-state machine (FSM) for planning and recovery (Guo et al. 2026). In contrast, CoAdapt-GUI uses FSM-grounded context to constrain which workflow knowledge may transfer across applications, excluding app-bound source state from transfer.

E-SPL jointly optimizes a global free-text prompt and a policy from shared rollouts (Zhang, Chen, and Stadie 2026). CoAdapt-GUI instead combines validated evolution of structured workflow context with reward-based LoRA adaptation. Their overlap is therefore limited to the high-level idea of joint context-policy adaptation.

3 CoAdapt-GUI: Test-Time Context-Policy Adaptation

We study test-time adaptation to an unseen GUI application. Our central premise is that test-time behavior depends on two complementary adaptive states. An explicit workflow state M_t provides contextual guidance about procedures, failure conditions, recovery strategies, and completion checks. A parametric state θ_t controls the policy’s underlying visual-action behavior. Rollouts in the target application provide executable feedback for adapting both states.

Co-adapting these states introduces two forms of interference. First, source experience may mix portable workflow knowledge with application-specific screens, identifiers, geometry, and navigation paths, causing negative transfer on an unseen interface. Meanwhile, changes to the workflow context alter the effective policy input, so reward differences across context conditions need not reflect differences in policy behavior.

We propose CoAdapt-GUI, which combines transfer-constrained workflow adaptation with task-context-matched policy learning. The context channel prevents app-bound source details from crossing application boundaries and stores target-grounded revisions separately. The policy channel computes relative advantages only among rollouts sharing the same task and context condition. The updates are coupled through interaction: the current workflow state shapes the trajectories used for policy learning, while the current policy generates the successes and failures used to revise workflow knowledge. Figure 1 summarizes the resulting adaptation loop.

3.1 Test-Time Adaptation Problem

Let $\mathcal{A}_{\text{src}}$ and $\mathcal{A}_{\text{tgt}}$ denote the source and target application sets. In the strict unseen-app setting, $\mathcal{A}_{\text{src}} \cap \mathcal{A}_{\text{tgt}} = \emptyset$. More generally, target trajectories never enter source initialization, and application-specific interaction knowledge from one application is not transferred to another.

For test-time adaptation, tasks are partitioned into disjoint adaptation and evaluation sets, $\mathcal{Q}^{\text{adapt}}$ and $\mathcal{Q}^{\text{eval}}$. Only tasks in $\mathcal{Q}^{\text{adapt}}$ may update the deployed agent; evaluation tasks and outcomes remain unavailable throughout adaptation.

The workflow state for task q combines an immutable source workflow context $M_0(q)$ with a target-grounded state M_t^{tgt} :

$$M_t(q) = (M_0(q), M_t^{\text{tgt}}), \quad C_t(q) = \text{Render}(M_t(q), q).$$

At interaction step k , the effective policy conditions on the task, current observation o_k , interaction history h_k , and rendered workflow context:

$$u_k \sim \pi_{\theta_t}(\cdot \mid q, o_k, h_k, C_t(q)).$$

Executing the action sequence produces a trajectory $\tau = \{(o_k, u_k)\}_{k=1}^K$ and a task reward $r(\tau)$.

The parametric state θ_t contains the trainable LoRA parameters of an otherwise frozen vision-language policy. Together with the target environment, $M_t(q)$ and θ_t induce the rollout distribution

$$\tau \sim d(\cdot \mid q, M_t(q), \theta_t).$$

Updating either state therefore changes the interaction data available for updating the other.

Given an interaction budget B , adaptation returns final states (M^*, θ^*) . Both are frozen before evaluation, and their generalization is estimated only on $\mathcal{Q}^{\text{eval}}$. Adaptation rewards are therefore training signals rather than the final evaluation objective.

3.2 Transfer-Constrained Workflow Context

Source trajectories mix reusable procedural knowledge with application-specific interaction details. Procedures, failure conditions, and completion checks may remain useful across applications with similar functionality, whereas screen layouts, visible strings, resource identifiers, coordinates, and source-specific navigation paths may cause negative transfer.

We represent each transferable workflow entry as

$$w = \langle c, P, F, V \rangle,$$

where c specifies when the workflow applies, P describes an abstract procedure, F records failure or recovery conditions, and V specifies observable or executable completion checks. These fields are stored in a typed schema so that applicability, execution guidance, failure handling, and verification can be validated and revised separately.

For each source application a , we construct a finite-state machine (FSM) grounded workflow context with two components: an app-bound state M_{app}^a and a transferable state M_{tr}^a . The app-bound state is instantiated as a screen-transition FSM recording concrete screens, action-conditioned transitions, visible interface cues, and resource-level information. This representation makes the expected effect of each action explicit, supporting the identification of failed transitions and possible recovery paths (Guo et al. 2026).

The transferable state contains workflow entries of the form defined above, abstracted from the same source trajectories used to construct the screen-transition FSM. Rather than encoding source-specific screens or transitions, it describes what should be accomplished, which failures should be avoided, and how task completion should be verified.

An eligibility predicate, $\text{Eligible}_{\text{tr}}(w) \in \{0, 1\}$, determines which entries may cross application boundaries. A schema validator and linter reject entries containing app names, package or resource identifiers, concrete widget labels, coordinates, task-instance values, and other app-bound content. Eligibility does not imply that an entry will help the target; it excludes identifiable source-specific information, while target rollout rewards subsequently determine utility.

A frozen synthesizer constructs the app-bound and transferable source states from audited trajectories. Let $\mathcal{W}_{\text{src}}$ denote the set of workflow entries w such that $w \in M_{\text{tr}}^a$ for some source application $a \in \mathcal{A}_{\text{src}}$ and $\text{Eligible}_{\text{tr}}(w) = 1$. These entries are grouped by functional category and consolidated into the source library

$$\mathcal{L}_{\text{src}} = \text{Consolidate}(\mathcal{W}_{\text{src}}).$$

For target task q , initialization retrieves bounded, eligibility-constrained protocol-permitted source workflow context, $M_0(q) = \text{Retrieve}(\mathcal{L}_{\text{src}}, q)$. Strict unseen-app evaluation excludes all app-bound source states. If the target belongs to a functional category absent from the source pool, retrieval returns an empty prior rather than forcing unrelated guidance.

The source library remains fixed during target adaptation. Target interactions instead maintain the separate target-grounded state M_t^{tgt} . Rendering selects task-relevant entries from the source workflow context and target-grounded state while preserving their provenance. This separation prevents target adaptation from overwriting reusable source knowledge while allowing target experience to refine or supplement the guidance used during execution.

3.3 Joint Context and Policy Adaptation

Target interactions can update both the workflow state and the policy parameters. At adaptation round t , the controller

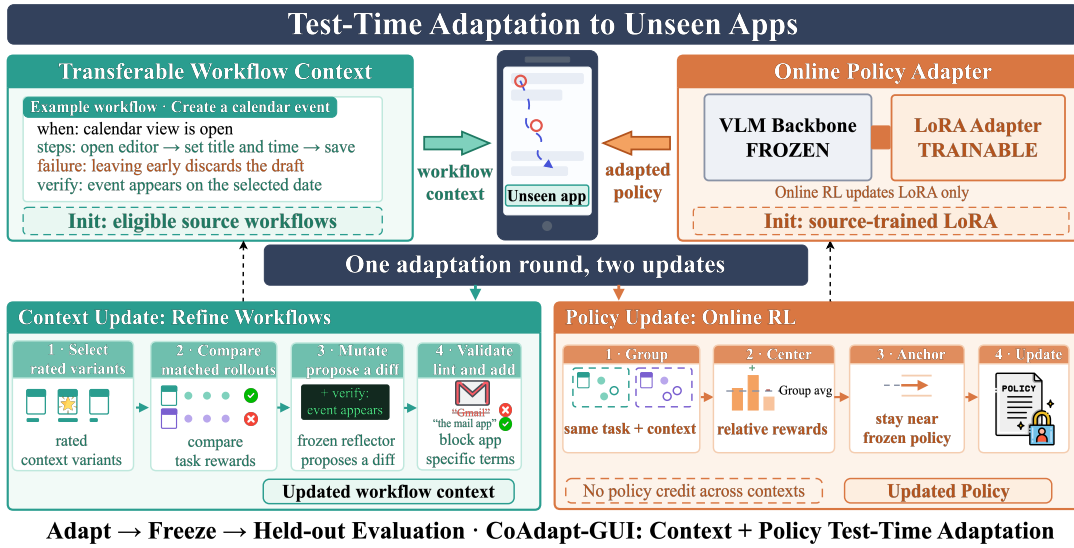


Figure 1: CoAdapt-GUI uses matched target-app rollouts to refine transferable workflow context and update a lightweight policy adapter on a frozen VLM. Context-Only TTA executes only the context branch, whereas CoAdapt-GUI executes both branches using the same rollout batches. The adapted context and policy are frozen before held-out evaluation.

selects a task $q_t \in \mathcal{Q}^{\text{adapt}}$ and one or more already materialized context variants from the current population. It then collects matched rollouts under controlled reset conditions:

$$D_t = \{(\tau_j, r_j, \kappa_j, z_j)\}_{j=1}^{N_t}.$$

Here, κ_j identifies the context variant that generated trajectory τ_j , and z_j records its reset identity. Only context variants present before collecting D_t can receive rating updates from their rewards. The trajectories are also added to the policy buffer $\mathcal{B}$ with their task and context identities.

The two channels reuse the same rollout stream but operate at different frequencies. Each matched batch updates the ratings of the evaluated context variants and may produce a validated child for subsequent evaluation. The LoRA parameters are updated only when $\mathcal{B}$ contains valid task-context comparison groups. The updates are therefore interaction-coupled rather than jointly differentiable: the current context shapes the trajectories used for policy learning, while the current policy determines the successful and failed behaviors available for future context revisions.

Reward-guided context update. For the repeated seeded tasks used in our evaluation, the controller maintains a population of TrueSkill-rated context variants (Herbrich, Minka, and Graepel 2006). At each round, it samples already materialized variants, evaluates them on matched tasks and resets seeds, and updates their ratings using the resulting task rewards. A frozen reflector then contrasts successful and failed traces from the evaluated variants and proposes a typed workflow revision to a high-rated parent.

The resulting child enters the population only after passing schema, provenance, and lint checks. Because the child is proposed after collecting the current round’s rollouts, it does not inherit their rewards. Its utility is determined only when it is sampled and evaluated in subsequent matched rollouts. At

the end of adaptation, the highest-rated validated workflow state is selected for deployment.

Task-context-matched policy update. The policy update learns online from trajectories generated by the current policy rather than from a fixed dataset. Because the context condition changes the effective policy input, trajectories are partitioned into groups sharing both task and context:

$$G(q, \kappa) = \{j \mid q_j = q, \kappa_j = \kappa\}.$$

For a group G of size n_G , we compute

$$\bar{r}_G = \frac{1}{n_G} \sum_{j \in G} r_j, \quad A_j = \frac{r_j - \bar{r}_G}{s_G},$$

where s_G is the normalization factor: it is set to one for mean-centered advantages, or to the within-group standard deviation plus a small constant for standardized advantages. Singleton and constant-reward groups provide no active policy gradient.

For trajectory τ_j , we average log-probabilities over its action-generation units:

$$\ell_j(\theta) = \frac{1}{|\tau_j|} \sum_{m \in \tau_j} \log \pi_\theta(u_{j,m} \mid x_{j,m}),$$

where $x_{j,m}$ contains the task, observation, history, and rendered context available when action unit $u_{j,m}$ is generated. Let $\mathcal{B}_{\text{act}} \subseteq \mathcal{B}$ contain trajectories belonging to non-degenerate groups. The resulting online group-relative objective (Shao et al. 2024) is

$$\mathcal{L}_{\text{policy}} = -\frac{1}{|\mathcal{B}_{\text{act}}|} \sum_{j \in \mathcal{B}_{\text{act}}} A_j \ell_j(\theta) + \beta \mathcal{R}(\theta; \pi_{\text{anchor}}),$$

where $\mathcal{R}$ denotes the frozen-policy regularizer when anchoring is enabled. The vision-language backbone remains

Algorithm 1: CoAdapt-GUI test-time context-policy adaptation

Require: Source workflow library $\mathcal{L}_{\text{src}}$, policy initialization θ_0 , adaptation tasks $\mathcal{Q}^{\text{adapt}}$, frozen reflector, interaction budget B

Ensure: Frozen workflow state M^* and policy parameters θ^*

- 1: Retrieve the protocol-permitted source workflow context M_0
- 2: Initialize target-grounded state $M^{\text{tgt}} \leftarrow \emptyset$
- 3: Initialize policy $\theta \leftarrow \theta_0$ and buffer $\mathcal{B} \leftarrow \emptyset$
- 4: **while** interaction budget remains **do**
- 5: Select a task $q \in \mathcal{Q}^{\text{adapt}}$
- 6: Select already materialized context variants for q
- 7: Collect matched rollouts D and record their task, context, and reset identities
- 8: Update the ratings of the evaluated variants using rewards in D
- 9: Propose and validate a child revision from traces in D
- 10: **if** the child passes all validation checks **then**
- 11: Add the child to the population for subsequent evaluation
- 12: **end if**
- 13: Add the same trajectories in D to $\mathcal{B}$
- 14: **if** LoRA adaptation is enabled and $\mathcal{B}$ is ready **then**
- 15: Update LoRA from nondegenerate groups in $\mathcal{B}$, if any
- 16: Clear $\mathcal{B}$
- 17: **end if**
- 18: **end while**
- 19: $M^* \leftarrow$ highest-rated validated workflow state
- 20: $\theta^* \leftarrow \theta$
- 21: Freeze (M^*, θ^*)
- 22: **return** (M^*, θ^*)

frozen, and only the LoRA parameters are optimized (Hu et al. 2021). Restricting comparisons to $G(q, \kappa)$ controls variation due to task difficulty and contextual guidance when computing relative advantages. The policy buffer $\mathcal{B}$ stores only trajectories collected since the most recent LoRA update. The policy remains fixed while these trajectories are collected. Once the update threshold is reached, at most one LoRA update is attempted, and the buffer is cleared, regardless of whether it contains an active comparison group. No trajectory is reused after the policy changes. Each trajectory retains the task and rendered-context identity under which it was collected.

Algorithm 1 summarizes the complete schedule. Each matched comparison evaluates only already materialized context variants; revisions proposed from its traces enter the population for subsequent evaluation. LoRA is updated only when the policy buffer contains nondegenerate task-context groups.

3.4 Source Initialization and Frozen Evaluation

During source preparation, the frozen synthesizer constructs the app-bound and transferable source states. Eligible workflow entries are consolidated into $\mathcal{L}_{\text{src}}$. Separately, a shared LoRA adapter is trained on source-app trajectories while workflow context is held fixed. The resulting context and policy initializations are independently usable: either may be deployed without enabling adaptation of the other.

At test time, initialization follows the evaluation protocol. Strict unseen-app evaluation excludes all app-bound source interaction knowledge, and a target from a novel functional category begins without a source workflow prior. The policy is initialized from a source-side checkpoint selected using source metadata. Target trajectories and held-out outcomes never influence source construction or checkpoint routing.

Algorithm 1 then uses only $\mathcal{Q}^{\text{adapt}}$. The source library remains immutable, while target interactions update the separate target-grounded workflow state and, when enabled, the LoRA parameters. Once the interaction budget is exhausted, the selected workflow state and final policy adapter are frozen. No task, trajectory, or reward from $\mathcal{Q}^{\text{eval}}$ may alter either state.

4 Experiments

We evaluate CoAdapt-GUI in two unseen-app settings. (1) **AndroidWorld-Generalization** follows the released setup and tests adaptation to new instances of task templates encountered during adaptation, enabling direct comparison with the reported Policy-Only TTA baseline (Gu et al. 2026). (2) **AndroidWorld Plus**, an experimental extension of AndroidWorld (Rawles et al. 2025), separates adaptation and evaluation by task template to test transfer to new task types within unseen apps.

4.1 Experimental Setup

Each experiment setting separates source data, target adaptation tasks, and held-out target evaluation tasks. Source data is used to obtain the initial policy and reusable workflow context. During target adaptation, the agent may update its workflow context, policy, or both, depending on the configuration. All updates stop before evaluation; no held-out task, trajectory, reward, or outcome is used for adaptation or checkpoint selection. A *task template* defines a reusable task type, whereas a *task instance* supplies its concrete arguments. Our primary metric is success rate (SR), defined as the percentage of held-out evaluation episodes successfully completed according to the benchmark’s programmatic evaluator.

Compared methods. We mainly compare five configurations in each setting. **The Base Policy** uses the initial policy without workflow context or a target-side update. **Static Context Transfer** supplies source workflows as frozen context. **Policy-Only TTA** keeps the workflow context fixed and updates only the policy adapter from target adaptation rollouts. **Context-Only TTA** keeps the policy fixed and builds additional workflow context from target adaptation rollouts. **CoAdapt-GUI** also updates a LoRA policy adapter while freezing the VLM backbone. These configurations isolate

source workflow transfer, target-side context adaptation, policy adaptation, and their joint use. Context-Only TTA and CoAdapt-GUI receive the same adaptation tasks and rollout budget, but are run independently and can therefore collect different policy-dependent trajectories; their difference reflects the complete joint procedure, not a controlled estimate of the LoRA update alone. Experiments were run on NVIDIA H200 GPUs with 141 GB of memory; one 20-round target-app adaptation run required approximately 9–10 GPU-hours on average.

4.2 New Task Instances in Unseen Apps

Setup. We follow the released unseen-app setting of AndroidWorld-Generalization (Gu et al. 2026). Its source split contains 12 apps, 62 task templates, and 905 training instances. All configurations start from the released step-500 UI-TARS-7B checkpoint (Qin et al. 2025), trained on this split. The target split contains five disjoint apps, with eight adaptation instances per app (40 total) and 48 held-out evaluation instances. Adaptation and evaluation cover the same 16 templates but use instances generated with non-overlapping seeds, thereby testing transfer to new instances of task types seen during adaptation.

We use the released manifests throughout. All adaptive methods use the same eight adaptation instances and nominal 50-step schedule per target app, and all methods are evaluated on the same 48 held-out instances. The Base Policy and Policy-Only TTA values are taken directly from AndroidWorld-Generalization (Gu et al. 2026).

Results. The reported Base Policy and Policy-Only TTA achieve 27.10% and 37.50%, respectively. Our CoAdapt-GUI configuration performs best at $45.00\% \pm 1.86$, 7.5 points above the reported Policy-Only TTA result. Context-Only TTA reaches $35.00\% \pm 1.74$, while Static Context Transfer reaches $28.75\% \pm 2.28$. Under the same adaptation instances and rollout budget, CoAdapt-GUI outperforms Context-Only TTA by 10.00 points, indicating that co-adapting workflow context and the policy is more effective than adapting context alone in this setting.

4.3 New Task Templates in Unseen Apps

Setup. To evaluate transfer to task templates not observed during adaptation, we construct AndroidWorld Plus by extending AndroidWorld (Rawles et al. 2025) with three apps from B-MoCA (Lee et al. 2026) and three from Android-Lab (Xu et al. 2024). After executable-task filtering, the benchmark contains 25 apps and 191 task templates. We assign 12 apps with 96 templates to the source set and the remaining 13 apps with 95 templates to the disjoint target set.

All AndroidWorld Plus configurations use Qwen3-VL-8B-Instruct (Bai et al. 2025) with a frozen backbone and start from a LoRA adapter trained only on the 12 source apps. Checkpoint selection occurs before target adaptation without using target rollouts or rewards, and each matched comparison uses the same initialization. Policy-Only TTA updates only the LoRA adapter while keeping its initial source context fixed. Supplementary Section 1 provides the filtering

procedure and complete app-level manifest.

Within each target app, adaptation and evaluation use disjoint task templates. Across the target apps, 60 templates yield a pool of 300 adaptation instances, while 35 held-out templates yield 105 evaluation episodes using non-overlapping seeds. Each adaptive configuration runs for 20 rounds per app with up to two context variants per round. Context-Only TTA, Policy-Only TTA, and CoAdapt-GUI collect four rollouts per selected task-context condition (≤ 160 per app). Context-Only TTA uses its batches only for workflow adaptation, Policy-Only TTA uses them only for LoRA adaptation, and CoAdapt-GUI uses them for both. Supplementary Section 3 provides the remaining adaptation details. This setting therefore tests whether experience from some task types transfers to unseen task types within the same target apps.

Source-category coverage. Following Android Control (Li et al. 2024), we use Google Play categories as a coarse, externally defined boundary for source-workflow retrieval. Source workflows are eligible only when their app category matches that of the target app. Under this protocol, six target apps are **Category-Shared** and can retrieve source workflows, whereas seven are **Category-Novel**, for which retrieval returns no source workflow. Static Context Transfer therefore supplies no workflow context for Category-Novel Apps. Context-Only TTA and CoAdapt-GUI instead initialize an empty target-grounded workflow state, construct workflow entries from target adaptation rollouts and rewards, and refine them in subsequent rounds. This breakdown assesses whether target-side adaptation remains effective without category-matched source experience. Supplementary Section 1 provides the complete app allocation and implementation details.

Results. Our CoAdapt-GUI performs best, raising overall success from 38.6% to 52.9%, a gain of 14.3 points. The cumulative context path—Static Context Transfer followed by Context-Only TTA—reaches 48.1%, 9.5 points above the Base Policy, whereas Policy-Only TTA reaches 40.0%, a gain of only 1.4 points. CoAdapt-GUI is a further 4.8 points above Context-Only TTA. This pattern is consistent with evolved workflow guidance producing more informative target experience for policy learning.

On Category-Shared Apps, CoAdapt-GUI reaches 70.4%, compared with 53.7% for Policy-Only TTA and 63.9% for Context-Only TTA. Category-Novel Apps provide a built-in control for source transfer. Because retrieval is empty in this group, Static Context Transfer exactly matches the Base Policy at 29.4%. Policy-Only TTA falls to 25.5%, whereas Context-Only TTA and CoAdapt-GUI improve success to 31.4% and 34.3%.

Separately, a representative Chrome adaptation trace illustrates how context evolution can affect the policy-learning signal. All evaluated variants received zero task reward during the first 11 rounds. In round 12, an evolved variant reached a mean task reward of 0.25 over four rollouts, while the root context remained at zero. Context evolution thus exposed a task-success signal that was not observed under the original workflow guidance.

Method	Config.	Adapt.	Steps/app	Eval.	Success Rate (%)	Source
The Base Policy (Gu et al. 2026)	no TTA	0	0	48	27.10	Reported
Static Context Transfer	static context	0	0	48	28.75 ± 2.28	Ours (5 runs)
Policy-Only TTA (Gu et al. 2026)	policy only	40	50/app	48	37.50	Reported
Context-Only TTA	context only	40	50/app	48	35.00 ± 1.74	Ours (5 runs)
CoAdapt-GUI	context + policy	40	50/app	48	45.00 ± 1.86	Ours (5 runs)

Table 1: **Setting 1—New Task Instances in Unseen Apps (AndroidWorld-Generalization)**. Success rates are measured on 48 held-out instances of task templates used during target adaptation. Reported results are from AndroidWorld-Generalization (Gu et al. 2026); our results are mean $\pm$ standard deviation over five runs.

Method	Config.	Adapt.	Steps/app	Category-Shared Apps	Category-Novel Apps	Overall SR
The Base Policy	no TTA	0	0	47.2%	29.4%	38.6%
Static Context Transfer	static context	0	0	56.5%	29.4%	43.3%
Policy-Only TTA	policy only	300	20/app	53.7%	25.5%	40.0%
Context-Only TTA	context only	300	20/app	63.9%	31.4%	48.1%
CoAdapt-GUI	context + policy	300	20/app	70.4%	34.3%	52.9%

Table 2: **Setting 2—New Task Templates in Unseen Apps (AndroidWorld Plus)**. Success rates are measured over 105 held-out episodes whose task templates are disjoint from target adaptation. Category-Shared and Category-Novel group episodes by whether the target app’s category is represented in the source pool.

5 Limitations

Evaluation scope and leakage control. Target-side adaptation can overestimate generalization through information leakage or evaluation on overly similar target tasks. We prevent leakage by using disjoint source and target app pools, restricting all updates and model selection to the adaptation split, and freezing both adapted states before held-out evaluation. We further evaluate two levels of task separation: AndroidWorld-Generalization uses disjoint instances generated from the same templates, whereas AndroidWorld Plus uses disjoint adaptation and evaluation templates within every target app. Fixed manifests, repeated runs, and a common evaluation harness are used across matched configurations.

Transfer and context reliability. Source workflows may contain interface-specific assumptions or errors introduced during reflection, causing negative transfer to a new application. CoAdapt-GUI addresses this risk in two stages. Before transfer, it separates app-bound screen-transition state from reusable workflow knowledge and applies eligibility, schema, provenance, and lint checks before a source workflow enters the transferable library. During target adaptation, a proposed revision first enters the candidate population and is evaluated in subsequent matched rollouts; task rewards update its rating and determine whether it is retained in the final workflow state.

Sparse feedback and adaptation stability. A limited target-interaction budget can produce noisy or uniform rewards, making it difficult to distinguish useful context revisions and assign policy credit. To reduce this ambiguity, we evaluate context variants on matched tasks and reset seeds, and compute policy advantages only among rollouts sharing the same task and context condition. Groups without reward variation are excluded from policy updates rather than assigned artificial credit. We also clear the on-policy buffer after each update attempt and restrict training to a LoRA adapter on a frozen backbone, reducing stale-policy updates

and limiting parameter drift under sparse supervision.

Deployment safety and continual use. Exploration on a new application may expose private information, trigger irreversible actions, or accumulate unstable updates over time. We therefore conduct adaptation in resettable emulators, restrict interaction to designated adaptation tasks and a fixed budget, and freeze the selected context and LoRA checkpoint before held-out use. App-bound workflow state remains local to its application, while only validated transferable knowledge may cross application boundaries. Updating only a lightweight LoRA adapter on a frozen backbone further limits parameter drift and the cost of repeated adaptation.

6 Conclusion

We introduced CoAdapt-GUI, a test-time adaptation framework that updates workflow context and policy on unseen applications using the agent’s own target rollouts and task rewards. Its transfer-constrained context retains reusable workflow knowledge while excluding app-bound source state, and its policy update assigns credit only within matched task–context conditions. CoAdapt-GUI reaches 45.0% on AndroidWorld-Generalization, compared with 37.5% for the reported Policy-Only TTA baseline, and improves AndroidWorld Plus from 38.6% to 52.9%. Context adaptation improves performance without policy updates in both settings, while CoAdapt-GUI achieves the strongest overall results, supporting context and policy adaptation as distinct but complementary channels for novel-app generalization.

References

Bai, S.; Cai, Y.; Chen, R.; Chen, K.; Chen, X.; Cheng, Z.; Deng, L.; Ding, W.; Gao, C.; Ge, C.; Ge, W.; Guo, Z.; Huang, Q.; Huang, J.; Huang, F.; Hui, B.; Jiang, S.; Li, Z.; Li, M.; Li, M.; Li, K.; Lin, Z.; Lin, J.; Liu, X.; Liu, J.; Liu, C.; Liu, Y.; Liu, D.; Liu, S.; Lu, D.; Luo, R.; Lv, C.; Men, R.; Meng, L.; Ren, X.; Ren, X.; Song, S.; Sun, Y.; Tang, J.; Tu, J.; Wan,

- J.; Wang, P.; Wang, P.; Wang, Q.; Wang, Y.; Xie, T.; Xu, Y.; Xu, H.; Xu, J.; Yang, Z.; Yang, M.; Yang, J.; Yang, A.; Yu, B.; Zhang, F.; Zhang, H.; Zhang, X.; Zheng, B.; Zhong, H.; Zhou, J.; Zhou, F.; Zhou, J.; Zhu, Y.; and Zhu, K. 2025. Qwen3-VL Technical Report. *arXiv:2511.21631*.
- Chen, A.; Liu, Z.; Zhang, J.; Prabhakar, A.; Liu, Z.; Heinecke, S.; Savarese, S.; Zhong, V.; and Xiong, C. 2026. Test-Time Adaptation for LLM Agents via Environment Interaction. In *The Fourteenth International Conference on Learning Representations*.
- Cheng, K.; Li, Z.; Ma, Z.; Chen, N.; Cao, J.; Sun, Q.; Ding, Z.; Xu, F.; Yan, H.; Chen, J.; Luu, A. T.; Zhang, J.; Lu, L.; and Lin, D. 2026. OpenMobile: Building Open Mobile Agents with Task and Trajectory Synthesis. *arXiv:2604.15093*.
- Cheng, K.; Sun, Q.; Chu, Y.; Xu, F.; Li, Y.; Zhang, J.; and Wu, Z. 2024. SeeClick: Harnessing GUI Grounding for Advanced Visual GUI Agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*.
- Deng, X.; Gu, Y.; Zheng, B.; Chen, S.; Stevens, S.; Wang, B.; Sun, H.; and Su, Y. 2023. Mind2Web: Towards a Generalist Agent for the Web. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*.
- Gou, B.; Wang, R.; Zheng, B.; Xie, Y.; Cheng, C.; Shu, Y.; Sun, H.; and Su, Y. 2024. Navigating the Digital World as Humans Do: Universal Visual Grounding for GUI Agents. *arXiv preprint arXiv:2410.05243*.
- Gu, L.; Jiang, Z.; Chi, Z.; Liu, H.; Wang, Z.; Yu, Y.; Berseth, G.; and Wang, Y. 2026. Generalization in Online Reinforcement Learning for Mobile Agents. *arXiv preprint arXiv:2603.07432*.
- Guo, L.; Liu, W.; Heng, Y. W.; Chen, T.; and Wang, Y. 2026. Agent-SAMA: State-Aware Mobile Assistant. In *Proceedings of the AAAI Conference on Artificial Intelligence*.
- Herbrich, R.; Minka, T.; and Graepel, T. 2006. TrueSkill™: A Bayesian Skill Rating System. In Schölkopf, B.; Platt, J.; and Hoffman, T., eds., *Advances in Neural Information Processing Systems*, volume 19. MIT Press.
- Hong, W.; Wang, W.; Lv, Q.; Xu, J.; Yu, W.; Ji, J.; Wang, Y.; Wang, Z.; Dong, Y.; Ding, M.; and Tang, J. 2024. CogAgent: A Visual Language Model for GUI Agents. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 14281–14290.
- Hu, E. J.; Shen, Y.; Wallis, P.; Allen-Zhu, Z.; Li, Y.; Wang, S.; Wang, L.; and Chen, W. 2021. LoRA: Low-Rank Adaptation of Large Language Models. *arXiv preprint arXiv:2106.09685*.
- Kong, Q.; Zhang, X.; Yang, Z.; Gao, N.; Liu, C.; Tong, P.; Cai, C.; Zhou, H.; Zhang, J.; Chen, L.; Liu, Z.; Hoi, S.; and Wang, Y. 2026. MobileWorld: Benchmarking Autonomous Mobile Agents in Agent-User Interactive and MCP-Augmented Environments. In Liakata, M.; Moreira, V. P.; Zhang, J.; and Jurgens, D., eds., *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 6142–6167. San Diego, California, United States: Association for Computational Linguistics. ISBN 979-8-89176-390-6.
- Lee, J.; Min, T.; An, M.; Hahm, D.; Lee, H.; Kim, C.; and Lee, K. 2026. Benchmarking Mobile Device Control Agents across Diverse Configurations. In Chandar, S.; Pascanu, R.; Eaton, E.; Liu, B.; Mahmood, R.; and Rannen-Triki, A., eds., *Proceedings of The 4th Conference on Lifelong Learning Agents*, volume 330 of *Proceedings of Machine Learning Research*, 261–306. PMLR.
- Li, W.; Bishop, W.; Li, A.; Rawles, C.; Campbell-Ajala, F.; Tyamagundlu, D.; and Riva, O. 2024. On the Effects of Data Scale on UI Control Agents. *arXiv preprint arXiv:2406.03679*.
- Liu, G.; Zhao, P.; Liu, L.; Chen, Z.; Chai, Y.; Ren, S.; Wang, H.; He, S.; and Meng, W. 2025a. LearnAct: Few-Shot Mobile GUI Agent with a Unified Demonstration Benchmark. *arXiv preprint arXiv:2504.13805*.
- Liu, Z.; Zhang, C.; Zhao, X.; Feng, Y.; Bai, B.; Feng, D.; Feng, E.; Xia, Y.; and Chen, H. 2025b. Beyond Training: Enabling Self-Evolution of Agents with MOBIMEM. *arXiv preprint arXiv:2512.15784*.
- Qin, Y.; Ye, Y.; Fang, J.; Wang, H.; Liang, S.; Tian, S.; Zhang, J.; Li, J.; Li, Y.; Huang, S.; Zhong, W.; Li, K.; Yang, J.; Miao, Y.; Lin, W.; Liu, L.; Jiang, X.; Ma, Q.; Li, J.; Xiao, X.; Cai, K.; Li, C.; Zheng, Y.; Jin, C.; Li, C.; Zhou, X.; Wang, M.; Chen, H.; Li, Z.; Yang, H.; Liu, H.; Lin, F.; Peng, T.; Liu, X.; and Shi, G. 2025. UI-TARS: Pioneering Automated GUI Interaction with Native Agents. *arXiv:2501.12326*.
- Rawles, C.; Clinckemaillie, S.; Chang, Y.; Waltz, J.; Lau, G.; Fair, M.; Li, A.; Bishop, W.; Li, W.; Campbell-Ajala, F.; Toyama, D.; Berry, R.; Tyamagundlu, D.; Lillicrap, T.; and Riva, O. 2025. AndroidWorld: A Dynamic Benchmarking Environment for Autonomous Agents. *arXiv:2405.14573*.
- Shao, Z.; Wang, P.; Zhu, Q.; Xu, R.; Song, J.; Bi, X.; Zhang, H.; Zhang, M.; Li, Y.; Wu, Y.; and Guo, D. 2024. DeepSeek-Math: Pushing the Limits of Mathematical Reasoning in Open Language Models. *arXiv preprint arXiv:2402.03300*.
- Sun, Y.; Wang, X.; Liu, Z.; Miller, J.; Efros, A. A.; and Hardt, M. 2020. Test-Time Training with Self-Supervision for Generalization under Distribution Shifts. In *International Conference on Machine Learning (ICML)*.
- Verma, G.; Kaur, R.; Srishankar, N.; Zeng, Z.; Balch, T.; and Veloso, M. 2024. AdaptAgent: Adapting Multimodal Web Agents with Few-Shot Learning from Human Demonstrations. *arXiv preprint arXiv:2411.13451*.
- Wang, D.; Shelhamer, E.; Liu, S.; Olshausen, B.; and Darrell, T. 2021. Tent: Fully Test-Time Adaptation by Entropy Minimization. In *International Conference on Learning Representations (ICLR)*.
- Wang, Z.; Xu, H.; Wang, J.; Zhang, X.; Yan, M.; Zhang, J.; Huang, F.; and Ji, H. 2025. Mobile-Agent-E: Self-Evolving Mobile Assistant for Complex Tasks. *arXiv preprint arXiv:2501.11733*.
- Wang, Z. Z.; Mao, J.; Fried, D.; and Neubig, G. 2024. Agent Workflow Memory. *arXiv preprint arXiv:2409.07429*.
- Xiao, H.; Wang, G.; Wang, H.; Liu, S.; Chai, Y.; Pan, Y.; Zhou, Y.; Chen, X.; Wen, Y.; and Li, H. 2026. UI-Mem: Self-Evolving Experience Memory for Online Reinforcement Learning in Mobile GUI Agents. *arXiv:2602.05832*.

Xu, Y.; Liu, X.; Liu, X.; Fu, J.; Zhang, H.; Jing, B.; Zhang, S.; Wang, Y.; Zhao, W.; and Dong, Y. 2025. MobileRL: Online Agentic Reinforcement Learning for Mobile GUI Agents. *arXiv:2509.18119*.

Xu, Y.; Liu, X.; Sun, X.; Cheng, S.; Yu, H.; Lai, H.; Zhang, S.; Zhang, D.; Tang, J.; and Dong, Y. 2024. AndroidLab: Training and Systematic Benchmarking of Android Autonomous Agents. *arXiv preprint arXiv:2410.24024*.

Zhang, C.; Yang, Z.; Liu, J.; Li, Y.; Han, Y.; Chen, X.; Huang, Z.; Fu, B.; and Yu, G. 2025. AppAgent: Multimodal Agents as Smartphone Users. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems, CHI '25*. New York, NY, USA: Association for Computing Machinery. ISBN 9798400713941.

Zhang, L.; Chen, R.; and Stadie, B. C. 2026. Evolutionary System Prompt Learning for Reinforcement Learning in LLMs. *arXiv preprint arXiv:2602.14697*.

Zuo, Y.; Zhang, K.; Sheng, L.; Qu, S.; Cui, G.; Zhu, X.; Li, H.; Zhang, Y.; Long, X.; Hua, E.; Qi, B.; Sun, Y.; Ma, Z.; Yuan, L.; Ding, N.; and Zhou, B. 2025. TTRL: Test-Time Reinforcement Learning. *arXiv preprint arXiv:2504.16084*.

Zweiger, A.; Pari, J.; Guo, H.; Akyürek, E.; Kim, Y.; and Agrawal, P. 2025. Self-Adapting Language Models. *arXiv preprint arXiv:2506.10943*.

Appendix / Supplementary

Appendix overview. This appendix reports the complete data allocation, implementation details, workflow-context examples, additional analyses, prompt templates, and validation contracts for CoAdapt-GUI. Section numbering is continuous within the appendix and cross-references are resolved in the main document.

A AndroidWorld Plus Construction and Data Allocation

A.1 Benchmark construction and executable filtering

AndroidWorld Plus is an experimental extension of AndroidWorld (Rawles et al. 2025). We retain the 19 AndroidWorld apps with app-attributable executable tasks and add six apps with their task suites: Calculator, Snapseed, and Wikipedia from B-MoCA (Lee et al. 2026), and Bluecoins, Maps.me, and Pi Music from AndroidLab (Xu et al. 2024). This produces 25 apps spanning 12 Google Play Store categories. We use the published Play Store category as an external functional taxonomy, following Android Control (Li et al. 2024), rather than defining categories from task outcomes.

The merged metadata contains 194 task rows. Two generic or composite tasks, `OpenApp` and `SaveCopyOfReceipt`, cannot be attributed to one app and are excluded from the app-level split, leaving 192 app-attributable templates. We then validate every template against the executable task registry. `WikipediaDecreaseTextSize50` is defined in the upstream task file but is not registered with the execution harness and therefore cannot be instantiated. We remove this template before freezing the manifests, yielding 191 executable task templates. No filtering decision uses an agent rollout, reward, or evaluation outcome.

A.2 Source and target app pools

The source and target app pools are disjoint. The source pool contains 12 apps and 96 templates (Table 3). Five task seeds per template, {30, 31, 32, 33, 34}, provide 480 source episodes for source-side policy training and workflow construction.

Source app	Category	Templates	Source app	Category	Templates
markor	Productivity	14	tasks_org	Productivity	6
joplin	Productivity	4	calculator	Tools	19
clock	Tools	3	files	Tools	2
bluecoins	Finance	15	pi_music	Music & Audio	12
audio_recorder	Music & Audio	2	snapseed	Photography	11
simple_sms_messenger	Communication	6	contacts	Communication	2
Total					96

Table 3: Complete AndroidWorld Plus source-app allocation. All source templates are excluded from the target pool.

The target pool contains the remaining 13 apps and 95 templates. For the category-coverage analysis, a target app’s group depends only on whether its category is represented in the source pool. Six target apps satisfy this condition and form the **Category-Shared Apps** group. The other seven belong to categories absent from the source pool and form the **Category-Novel Apps** group. Table 4 gives the complete target allocation.

A.3 Template and seed separation

Within each target app, we sort the executable template identifiers and create disjoint adaptation and evaluation sets. For an app with at least three templates, the first $\lceil 0.6n \rceil$ templates are assigned to adaptation and the remainder to evaluation. Two-template apps use a one-one split; a single-template app contributes only to adaptation. This deterministic rule produces 60 adaptation templates and 35 evaluation templates, with no template overlap in any target app. The split was frozen before target rollout collection and does not use task rewards, policy predictions, or evaluation outcomes.

Each adaptation template is instantiated with seeds {30, 31, 32, 33, 34}, defining a 300-instance sampling pool. Each evaluation template is instantiated with the disjoint seeds {40, 41, 42}, producing the frozen 105-episode panel used for all reported AndroidWorld Plus results. The 300 adaptation instances specify the available sampling pool rather than the number of target rollout calls: CoAdapt-GUI samples from this pool under the fixed interaction schedule described in Appendix C.

A.4 Complete source and target template manifests

Tables 5 and 6 list all 191 executable templates in the frozen AndroidWorld Plus manifest. Source templates are used only for source-side policy training and workflow construction. Target templates are partitioned into adaptation and held-out evaluation sets within each target app. Identifiers match the execution harness exactly.

Group	Target app	Category	Templates		Episodes	
			Adapt.	Eval.	Adapt. pool	Eval.
Category-Shared	simple_calendar_pro	Productivity	11	6	55	18
Category-Shared	system_settings	Tools	9	6	45	18
Category-Shared	pro_expense	Finance	6	3	30	9
Category-Shared	retro_music	Music & Audio	3	1	15	3
Category-Shared	camera	Photography	1	1	5	3
Category-Shared	chrome	Communication	2	1	10	3
<i>Category-Shared subtotal</i>			32	18	160	54
Category-Novel	maps_me	Maps & Navigation	9	6	45	18
Category-Novel	osmand	Maps & Navigation	2	1	10	3
Category-Novel	broccoli	Food & Drink	8	5	40	15
Category-Novel	opentracks	Health & Fitness	4	2	20	6
Category-Novel	wikipedia	Books & Reference	3	2	15	6
Category-Novel	vlc	Video Players & Editors	1	1	5	3
Category-Novel	simple_draw_pro	Art & Design	1	0	5	0
<i>Category-Novel subtotal</i>			28	17	140	51
Target total			60	35	300	105

Table 4: Complete AndroidWorld Plus target-app allocation. The adaptation pool uses five seeds per adaptation template; frozen evaluation uses three disjoint seeds per evaluation template. `simple_draw_pro` has one executable template and is used only for adaptation, so the held-out panel covers 12 of the 13 target apps.

Table 5: Complete AndroidWorld Plus source-pool template manifest

Source app	Category	Count	Executable source templates
markor	Productivity	14	• MarkorAddNoteHeader • MarkorChangeNoteContent • MarkorCreateFolder • MarkorCreateNote • MarkorCreateNoteAndSms • MarkorCreateNoteFromClipboard • MarkorDeleteAllNotes • MarkorDeleteNewestNote • MarkorDeleteNote • MarkorEditNote • MarkorMergeNotes • MarkorMoveNote • MarkorTranscribeReceipt • MarkorTranscribeVideo
tasks_org	Productivity	6	• TasksCompletedTasksForDate • TasksDueNextWeek • TasksDueOnDate • TasksHighPriorityTasks • TasksHighPriorityTasksDueOnDate • TasksIncompleteTasksOnDate
joplin	Productivity	4	• NotesIsTodo • NotesMeetingAttendeeCount • NotesRecipeIngredientCount • NotesTodoItemCount

Continued on the next page

Complete AndroidWorld Plus source-pool template manifest (continued)

Source app	Category	Count	Executable source templates
calculator	Tools	19	• CalculatorConvert45DegreesToRadians • CalculatorGeometricMean • CalculatorHarmonicMean • CalculatorInput1 • CalculatorInput10Choose2 • CalculatorInput17Times23 • CalculatorInput1Plus1 • CalculatorInput2Plus24Div3 • CalculatorInput3Times5 • CalculatorInput5Choose2 • CalculatorInputCos180 • CalculatorInputCos60 • CalculatorInputFactorial6 • CalculatorInputLn1234 • CalculatorInputPercent50Of28 • CalculatorInputSqrt25 • CalculatorOpen • CalculatorSumFirst5Fibonacci • CalculatorSumFirst5Primes
clock	Tools	3	• ClockStopWatchPausedVerify • ClockStopWatchRunning • ClockTimerEntry
files	Tools	2	• FilesDeleteFile • FilesMoveFile
bluecoins	Finance	15	• BluecoinsAddExpense • BluecoinsAddExpenseOnDate • BluecoinsAddExpenseOnDateWithLabel • BluecoinsAddIncomeOnDateWithNote • BluecoinsAddIncomeWithLabel • BluecoinsEditExpenseAmount • BluecoinsEditExpenseDateAmountNote • BluecoinsEditIncomeDateAndAmount • BluecoinsEditTransactionType • BluecoinsEditTransactionTypeAmountNote • BluecoinsQueryCategorySpending • BluecoinsQuerySpendingCategory • BluecoinsQuerySpendingOnDate • BluecoinsQueryTotalSpendingOnDate • BluecoinsQueryTransactionCount
pi_music	Music & Audio	12	• PiMusicCreatePlaylist • PiMusicPauseAndSeek • PiMusicPlayFromPlaylist • PiMusicPlaySongByTitleArtist • PiMusicQueryArtistSongCount • PiMusicQueryArtistTotalDuration • PiMusicQueryLongestSongDuration • PiMusicQuerySongAlbum • PiMusicQuerySortedSongsByTitle • PiMusicQueryTotalSongs • PiMusicSortByDurationAscending • PiMusicSortByDurationDescending
audio_recorder	Music & Audio	2	• AudioRecorderRecordAudio • AudioRecorderRecordAudioWithFileName

Continued on the next page

Complete AndroidWorld Plus source-pool template manifest (continued)

Source app	Category	Count	Executable source templates
snapseed	Photography	11	• SnapseedTask1 • SnapseedTask10 • SnapseedTask11 • SnapseedTask2 • SnapseedTask3 • SnapseedTask4 • SnapseedTask5 • SnapseedTask6 • SnapseedTask7 • SnapseedTask8 • SnapseedTask9
simple_sms_me ssenger	Communication	6	• SimpleSmsReply • SimpleSmsReplyMostRecent • SimpleSmsResend • SimpleSmsSend • SimpleSmsSendClipboardContent • SimpleSmsSendReceivedAddress
contacts	Communication	2	• ContactsAddContact • ContactsNewContactDraft

Table 6: Complete AndroidWorld Plus target adaptation/evaluation manifest

Adaptation templates	Held-out evaluation templates
simple_calendar_pro <i>Category-Shared; Productivity</i> 11 templates • SimpleCalendarAddOneEvent • SimpleCalendarAddOneEventInTwoWeeks • SimpleCalendarAddOneEventRelativeDay • SimpleCalendarAddOneEventTomorrow • SimpleCalendarAddRepeatingEvent • SimpleCalendarAnyEventsOnDate • SimpleCalendarDeleteEvents • SimpleCalendarDeleteEventsOnRelativeDay • SimpleCalendarDeleteOneEvent • SimpleCalendarEventOnDateAtTime • SimpleCalendarEventsInNextWeek	6 templates • SimpleCalendarEventsInTimeRange • SimpleCalendarEventsOnDate • SimpleCalendarFirstEventAfterStartTime • SimpleCalendarLocationOfEvent • SimpleCalendarNextEvent • SimpleCalendarNextMeetingWithPerson
system_settings <i>Category-Shared; Tools</i> 9 templates • SystemBluetoothTurnOff • SystemBluetoothTurnOffVerify • SystemBluetoothTurnOn • SystemBluetoothTurnOnVerify • SystemBrightnessMax • SystemBrightnessMaxVerify • SystemBrightnessMin • SystemBrightnessMinVerify • SystemCopyToClipboard	6 templates • SystemWifiTurnOff • SystemWifiTurnOffVerify • SystemWifiTurnOn • SystemWifiTurnOnVerify • TurnOffWifiAndTurnOnBluetooth • TurnOnWifiAndOpenApp

Continued on the next page

Adaptation templates	Held-out evaluation templates
pro_expense <i>Category-Shared; Finance</i> 6 templates - ExpenseAddMultiple - ExpenseAddMultipleFromGallery - ExpenseAddMultipleFromMarkor - ExpenseAddSingle - ExpenseDeleteDuplicates - ExpenseDeleteDuplicates2	3 templates - ExpenseDeleteMultiple - ExpenseDeleteMultiple2 - ExpenseDeleteSingle
retro_music <i>Category-Shared; Music & Audio</i> 3 templates - RetroCreatePlaylist - RetroPlayingQueue - RetroPlaylistDuration	1 template RetroSavePlaylist
camera <i>Category-Shared; Photography</i> 1 template CameraTakePhoto	1 template CameraTakeVideo
chrome <i>Category-Shared; Communication</i> 2 templates - BrowserDraw - BrowserMaze	1 template BrowserMultiply
maps_me <i>Category-Novel; Maps & Navigation</i> 9 templates - MapsMeAddWorkPlace - MapsMeCheckDrivingDistanceTime - MapsMeCheckNearestHotel - MapsMeCheckNearestPlace - MapsMeCheckNearestPlaceDriveTime - MapsMeCheckNearestPlaceWalkTime - MapsMeCheckPublicTransportRoute - MapsMeCheckRidingTime - MapsMeCheckWalkingDistanceTime	6 templates - MapsMeCompareRidingVsPublicTransport - MapsMeNavigateToBerkeley - MapsMeNavigateToLocation - MapsMeNavigateToOpenAI - MapsMeNavigateToStanford - MapsMeNavigateToUniversitySouth
osmand <i>Category-Novel; Maps & Navigation</i> 2 templates - OsmAndFavorite - OsmAndMarker	1 template OsmAndTrack
broccoli <i>Category-Novel; Food & Drink</i> 8 templates - RecipeAddMultipleRecipes - RecipeAddMultipleRecipesFromImage - RecipeAddMultipleRecipesFromMarkor - RecipeAddMultipleRecipesFromMarkor2 - RecipeAddSingleRecipe - RecipeDeleteDuplicateRecipes - RecipeDeleteDuplicateRecipes2 - RecipeDeleteDuplicateRecipes3	5 templates - RecipeDeleteMultipleRecipes - RecipeDeleteMultipleRecipesWithConstraint - RecipeDeleteMultipleRecipesWithNoise - RecipeDeleteSingleRecipe - RecipeDeleteSingleWithRecipeWithNoise
opentracks <i>Category-Novel; Health & Fitness</i> 4 templates - SportsTrackerActivitiesCountForWeek - SportsTrackerActivitiesOnDate - SportsTrackerActivityDuration - SportsTrackerLongestDistanceActivity	2 templates - SportsTrackerTotalDistanceForCategoryOverInterval - SportsTrackerTotalDurationForCategoryThisWeek

Continued on the next page

Adaptation templates	Held-out evaluation templates
wikipedia <i>Category-Novel; Books & Reference</i> 3 templates • WikipediaDisablePreviewAndFeed • WikipediaGoToSavedTab • WikipediaGoToSearchTab	2 templates • WikipediaIncreaseTextSize180 • WikipediaOpen
vlc <i>Category-Novel; Video Players & Editors</i> 1 template • VlcCreatePlaylist	1 template • VlcCreateTwoPlaylists
simple_draw_pro <i>Category-Novel; Art & Design</i> 1 template • SimpleDrawProCreateDrawing	0 templates <i>None</i>

B AndroidWorld-Generalization Protocol and Run Accounting

B.1 Released source and target splits

Setting 1 follows the released AndroidWorld-Generalization unseen-app split (Gu et al. 2026). Its source split contains 12 apps, 62 task templates, and 905 training instances. Every configuration starts from the released step-500 UI-TARS-7B checkpoint (Qin et al. 2025), which was trained only on this source split. The reusable source workflow library used by CoAdapt-GUI is likewise constructed only from this source split.

The target split contains five apps absent from the source split. It provides eight adaptation instances per app, for 40 in total, and 48 held-out evaluation instances. Adaptation and evaluation cover the same 16 task templates but instantiate them with non-overlapping seeds. This setting therefore evaluates transfer to new instances of task types encountered during target adaptation, rather than transfer to new task templates.

Target app	Task templates (released task ID)	Adapt. inst.	Eval. inst.
Audio Recorder	AudioRecorderRecordAudioWithFileName (28)	8	3
Clock	ClockTimerEntry (20)	8	3
OsmAnd	OsmAndFavorite (74); OsmAndMarker (88)	8	6
Tasks	TasksDueOnDate (100); TasksHighPriorityTasksDueOnDate (102); TasksCompletedTasksForDate (104); TasksIncompleteTasksOnDate (105)	8	12
Broccoli	RecipeDeleteMultipleRecipes (3); RecipeDeleteSingleRecipe (4); RecipeAddSingleRecipe (31); RecipeDeleteSingleWithRecipeWithNoise (32); RecipeAddMultipleRecipes (56); RecipeDeleteMultipleRecipesWithNoise (57); RecipeAddMultipleRecipesFromMarkor2 (82); and RecipeDeleteMultipleRecipesWithConstraint (90)	8	24
Total	16 templates	40	48

Table 7: Complete target allocation for AndroidWorld-Generalization. Each template is present in both splits, but the concrete task instances are disjoint.

B.2 Instance seeds and leakage control

The released evaluation manifest instantiates every target template with seeds {7, 30, 1234}. Adaptation uses different seeds: Audio Recorder and Clock use {1, 2, 3, 4, 5, 6, 8, 9}; each OsmAnd template uses {1, 2, 3, 4}; each Tasks template contributes two instances; and each Broccoli template contributes one instance. The released Tasks instances use seeds {1, 2}, except TasksCompletedTasksForDate, which uses {2, 3}. Seven Broccoli templates use seed 1 and RecipeDeleteSingleWithRecipeWithNoise uses seed 2. Consequently, the intersection between adaptation and evaluation (task_id, seed) pairs is empty.

All context revisions, policy updates, and checkpoint selection use only the 40-instance adaptation manifest. The workflow context and policy adapter are then frozen before the 48-instance evaluation manifest is opened. No held-out task instance, trajectory, reward, or outcome is used for adaptation or model selection.

B.3 Update schedule and result provenance

The adaptive configurations use the released eight adaptation instances per target app and a nominal 50-update schedule per app. They are evaluated on the same frozen 48-instance manifest. The schedule fixes the number of method-level updates; because context revision and policy optimization use different update operators, it should not be interpreted as an identical gradient or compute budget across methods.

The Base Policy and Policy-Only TTA values in the main paper are reported results from AndroidWorld-Generalization (Gu et al. 2026) and were not reproduced in our environment. Static Context Transfer, Context-Only TTA, and CoAdapt-GUI are our results. Each is evaluated in five complete runs, and the main paper reports the mean and standard deviation over those runs. All five runs use the same frozen adaptation and evaluation manifests; no incomplete run is included in the reported statistics.

C CoAdapt-GUI Implementation Details

This section specifies the realization of the context and policy branches in Algorithm 1 of the main paper. It describes CoAdapt-GUI itself rather than restating the implementation of every comparison configuration. The notation `layer1` and `layer2` below refers to the runtime JSON schema: `layer1` is the app-bound screen-transition state, and `layer2` is the transferable workflow state described in the paper.

C.1 Policy and source initialization

For AndroidWorld-Generalization, CoAdapt-GUI starts from the released step-500 UI-TARS-7B checkpoint and uses the released 50-step target schedule described in Appendix B. For AndroidWorld Plus, the acting policy is Qwen3-VL-8B-Instruct at revision `0c351dd01ed87e9c1b53cbc748cba10e6187ff3b`. The VLM backbone is loaded in `bfloat16` and remains frozen. We initialize it with a rank-16 LoRA adapter trained only on the 12 source apps. All configurations within a matched comparison use the same source-trained initialization, and configurations without policy adaptation keep it frozen. No target rollout, reward, or held-out outcome is used to construct or select this initialization.

Source workflow construction is independent of source-side LoRA training. A frozen synthesizer converts audited source trajectories into one app-bound screen-transition state and one transferable workflow state per source app. Only validated transferable entries are grouped by the externally defined Play Store category and consolidated into the immutable source library. App-bound screen states, transitions, UI strings, and identifiers never enter this library.

Table 8 reports the resulting frozen library. A target app can retrieve only the single library matching its Play Store category; cross-category retrieval is disabled. Thus a Category-Novel App receives no source entry, rather than an entry selected by semantic similarity or target reward.

Source category	Source apps	Workflow entries
Productivity	3	14
Tools	3	15
Finance	1	7
Music & Audio	2	11
Photography	1	6
Communication	2	7
Total	12	60

Table 8: Composition of the frozen transferable source-workflow library used by AndroidWorld Plus. Counts refer to validated structured workflow entries, not source trajectories or task templates.

C.2 AndroidWorld Plus adaptation configuration

Table 9 records the final configuration used for the AndroidWorld Plus experiments. The four rollouts associated with one task-context condition form the group size $N = 4$; N is not the number of context variants. At most $M = 2$ already materialized variants are selected in a round, so 20 rounds require at most $20 \times 2 \times 4 = 160$ target rollout calls per app. Early rounds may use only the root variant and therefore consume fewer calls.

C.3 Context population and reward-guided revision

Each target app has an independent population of transferable workflow states. For a Category-Shared App, the root contains the retrieved category-matched source workflow context. For a Category-Novel App, source retrieval returns empty and the target-grounded root is initialized empty. In the latter case, the first validated child constructs an initial transferable entry from target rollout evidence; later children refine that entry in the same way as in the nonempty case.

At round t , the controller samples up to two variants from the latest 15-member population window. Sampling is proportional to

Component	Parameter	Final value
Policy	Backbone / numerical precision	Qwen3-VL-8B-Instruct / bfloat16
Policy	Maximum image pixels / generated tokens	1,605,632 / 512
Policy	LoRA rank / scale / configured dropout	16/32/0.0
Policy	LoRA target modules	q_proj, v_proj
Policy	Optimizer / learning rate	AdamW / 3×10^{-4}
Policy	Maximum gradient norm	1.0
Policy	Advantage normalization	mean-centered within task-context group
Policy	Minimum active trajectories for an update	3
Policy	Frozen-policy anchor coefficient / log-ratio clip	0.05/10.0
Context	Initial TrueSkill mean / standard deviation	25.0/8.33
Context	TrueSkill performance / dynamics parameters	4.17/0.083
Context	Population window / child uncertainty increment	15/1.5
Context	Selection optimism / softmax temperature	1.0/1.0
Context	Frozen synthesizer and reflector	Claude Opus 4.7
Context	Reflection / diff output limits	4,096/32,000 tokens
Context	Malformed-output retries	3
Protocol	Adaptation rounds per target app	20
Protocol	Context variants per round	$M \leq 2$
Protocol	Rollouts per task-context condition	$N = 4$
Protocol	Maximum rollout calls per app	160
Protocol	Adaptation / evaluation task seeds	$\{30, 31, 32, 33, 34\} / \{40, 41, 42\}$
Protocol	Hardware / mean adaptation time	NVIDIA H200 141 GB / 9–10 GPU-hours

Table 9: CoAdapt-GUI implementation details for AndroidWorld Plus. The policy buffer is transient and is cleared after every update attempt; its lifetime is specified below.

$$p(i) \propto \exp\left(\frac{\mu_i + \lambda\sigma_i}{T}\right), \quad \lambda = 1, \quad T = 1,$$

where (μ_i, σ_i) is the variant’s TrueSkill rating. Selected variants are evaluated on matched task and reset conditions. Their benchmark task rewards determine their ordering and update their ratings. A frozen reflector then contrasts the resulting success and failure traces and proposes a typed revision to a high-rated parent.

The temporal order is important. Only variants that existed before the current rollouts receive ratings from those rollouts. A newly proposed child is added only after typed-diff parsing, transferable-state scope filtering, and a nonempty state change; the caller records its parent, task, iteration, and revision provenance. The child receives no inherited reward. Its usefulness is measured only if it is sampled in a later round. At budget exhaustion, the highest-rated validated state is selected as M^* .

The reflection record contains the task template and instantiated goal, reset seed, executable task reward, and a compact trace of actions, action reasons, post-action summaries, UI evidence, and parse or execution status. The caller retains at most ten UI-element lines per step, clips each line to 160 characters, and clips reasons and summaries to 400 characters. These limits bound the reflector input without replacing executable rewards with model judgments.

C.4 Task-context-matched LoRA update

Every trajectory is stored with its task, rendered-context identity, reset identity, and the exact multimodal inputs and action tokens used during collection. Policy advantages are computed only within groups whose members share the same task and context variant. In the reported AndroidWorld Plus configuration, each such group contains four rollouts and uses the mean-centered advantage

$$A_j = r_j - \frac{1}{|G|} \sum_{i \in G} r_i.$$

Singleton or constant-reward groups have zero advantage and are excluded from the active policy batch. An update is attempted only when at least three active trajectories remain. For each active trajectory, action-token log-probabilities are averaged within the trajectory before weighting by A_j , preventing a long rollout from dominating solely because it contains more action units.

The policy buffer is a transient on-policy accumulation buffer, not a replay memory. The policy remains fixed while a matched batch is collected. After the batch becomes eligible, at most one LoRA update is attempted and the buffer is cleared whether or not the batch contains an active reward comparison. No trajectory is reused after the policy changes. The frozen source-trained adapter supplies the policy anchor; the numerical log-ratio is clipped to $[-10, 10]$, and the final gradient norm is clipped to 1.0. Adapter dropout is disabled during both collection and gradient replay so that the saved behavior probabilities and the replayed current-policy probabilities refer to the same stochastic policy. Policy-sampling seeds, rather than dropout, provide the within-condition rollout diversity.

C.5 Frozen evaluation

After the interaction budget is exhausted, the selected workflow state and final LoRA adapter are written as immutable artifacts. Evaluation loads only these artifacts and the held-out manifest; the reflector, population controller, and optimizer are disabled. The structural, transferability, and behavioral checks applied before this point are specified together with the operative prompts in Appendix F.

D Workflow-Context Representation

This section gives concrete examples of the structured context described in the main paper. Each transferable workflow entry follows $w = \langle c, P, F, V \rangle$: an applicability condition, an abstract procedure, failure or recovery conditions, and completion checks. Each example below is condensed from a validated workflow artifact by removing repeated safeguards and provenance metadata; no new procedural, failure, or verification rule is introduced. App names are shown in the headings to identify the originating run, but are not part of the transferable text injected into the acting policy.

D.1 App-bound and transferable components

Table 10 illustrates the transfer boundary with a source artifact from `markor`. The app-bound component is a genuine screen-transition FSM: it includes concrete state names, visible strings, resource hints, and action-conditioned transitions. The transferable component retains the task logic while removing those interface bindings. Under unseen-app adaptation, only the right-hand form is eligible to initialize another app.

App-bound screen-transition state (not transferred)		Transferable workflow entry (eligible)
States: <code>file_browser</code> , identified by the Markor title, a red bottom-right create button, and the Files/To-Do/QuickNote/More navigation; <code>new_file_dialog</code> , identified by the pre-filled <code>my_note</code> field, format selector, and FOLDER/CANCEL/OK controls; and <code>editor_edit_mode</code> , identified by the editor toolbar and save icon.		Applicability: a list or index exposes an affordance for creating an item.
Transitions:		Procedure: invoke the create affordance; clear any placeholder; provide the required identifier; select the requested item type if needed; confirm creation; enter the requested content; and persist the change.
State transition	Action	Failures: retaining a placeholder in the identifier, choosing the wrong item type, creating in the wrong parent container, or declaring success before persistence.
<code>file_browser → new_file_dialog</code>	<code>click(fab_plus)</code>	Verification: the new item appears under the expected parent with the exact identifier, and reopening it shows the requested content.
<code>new_file_dialog → editor_edit_mode</code>	<code>click(OK)</code>	

Table 10: Concrete separation between app-bound interaction state and an app-agnostic workflow entry. The left column may support execution within its source app but is excluded from cross-application retrieval.

D.2 Representative highly rated target contexts

We next show three entries from highly rated validated states produced during AndroidWorld Plus adaptation. They cover both initialization regimes: the calendar example refines retrieved source knowledge for a Category-Shared App, whereas the navigation and activity-history examples were constructed from target rollouts after Category-Novel retrieval returned an empty source state. These entries are selected to illustrate the representation, not as additional quantitative evaluation examples.

Example 1: Scheduled-entry creation (Category-Shared)

Origin. The target app is `simple_calendar_pro`; its Productivity category is represented in the source pool. The entry starts from retrieved source workflow knowledge and is revised using target-app traces.

Applicability (*c*). A create form exposes multiple structured fields, such as title, date, start time, end time or duration, and optional content.

Procedure (*P*). Enumerate every goal-specified field before acting; resolve relative dates to absolute dates; set and visually confirm each field; after confirming the start time, derive the dependent end time; verify which picker component is active before editing it; save; then reopen the created item and read back every requested field.

Failure/recovery (*F*). Do not assume that an end-time field inherits an edited start time. Do not mistake an hour-to-minute auto-advance within one picker for a transition to another field. If the reasoning and action summary contain contradictory numeric values, re-inspect the form before continuing.

Verification (*V*). The saved detail view, rather than a potentially truncated list row, must show the requested date, start and end times, and other specified fields before completion is emitted.

Example 2: Nearest-place and route lookup (Category-Novel)

Origin. The target app is `maps_me`; Maps & Navigation is absent from the source pool. Its root transferable state is empty, and the entry is synthesized and subsequently refined from target adaptation traces.

Applicability (c). The task asks for a nearby place, place details, or a route with a specified travel mode.

Procedure (P). Establish a valid location and populated map view before searching; clear the query field and use a canonical place-category term; inspect both result rows and map markers; open a candidate’s detail view; for routing, set both endpoints and the requested travel mode before reading the rendered distance or duration.

Failure/recovery (F). A viewport-scoped empty result is not global infeasibility. Recenter or widen the map before reformulating the query. Clear existing text before retrying to avoid concatenated queries, and avoid random panning without a location reference.

Verification (V). A location reference and populated map are visible; the selected result’s type matches the goal; and any reported place, distance, or duration is read from the rendered result or route summary.

Example 3: Historical activity aggregation (Category-Novel)

Origin. The target app is `opentracks`; Health & Fitness is absent from the source pool. The selected workflow descends from a child that constructed the category entry from an empty root.

Applicability (c). A history query specifies a time window, an activity type, and an aggregation such as count, total, maximum, or average.

Procedure (P). Anchor relative dates to an observed current date; resolve the requested time interval and week convention; use available date and activity filters; inspect the full in-window set; disambiguate activity types using structured metadata rather than user-authored titles; aggregate the confirmed entries; and format the result in the requested unit and precision.

Failure/recovery (F). Avoid first-match answers for maximum or total queries, title-based activity classification, confusion between calendar weeks and rolling seven-day windows, and unit conversion based on rounded summary values when a precise detail value is available.

Verification (V). Every included entry has an observed type and timestamp inside the requested interval; the list boundary has been reached; units and time formats have been checked; and exactly one answer is followed by one completion action.

D.3 A recorded context-evolution step

The Chrome adaptation log provides a compact example of reward-guided context selection. The retrieved Communication context initially contained generic navigation and repeated-action procedures. A child proposed after target traces added three pieces of guidance: handle first-launch gates before continuing the task, verify that a post-action observation has actually advanced, and change strategy after repeated no-op actions. Table 11 shows that the child was admitted without inheriting its parent’s reward and was evaluated only in later rounds.

Round	Observed reward	Context event
6	root: 0.00	<code>gen1_mut_1</code> is proposed, validated, and admitted without an assigned reward.
7–11	all: 0.00	The child and existing alternatives are tested, but every evaluated condition remains at zero.
12	child: 0.25; root: 0.00	Under the matched <code>BrowserMaze</code> condition, the evolved context exposes nonzero executable feedback absent under the root.

Table 11: A recorded Chrome context-evolution event. Values are mean executable task rewards over the four rollouts for the indicated task–context condition.

The round-12 rating update favored the evolved branch, and later revisions retained its onboarding and state-advancement checks. The trace illustrates the intended temporal separation: a typed revision is first validated, then tested in subsequent rollouts, and retained according to executable task feedback.

E Additional Analyses and Ablations

E.1 Incremental contributions of the adaptation channels

Table 12 rewrites the AndroidWorld Plus results from the main paper as increments between the configurations that isolate each adaptation channel. Static Context Transfer improves Category-Shared Apps by 9.3 percentage points but leaves Category-Novel Apps unchanged, as expected from the category-restricted retrieval rule. Policy-Only TTA provides a 1.4 point overall gain but decreases Category-Novel performance. In contrast, target-side context construction improves both groups over Static Context Transfer, and enabling the complete joint procedure adds a further 4.8 points overall over Context-Only TTA.

These increments support complementary roles for the two channels. Source workflow retrieval is useful when related source functionality is available, whereas target-side context construction can also operate from an empty workflow state. Policy adaptation is most effective when combined with the evolved context. Because Context-Only TTA and CoAdapt-GUI

Increment	Category-Shared	Category-Novel	Overall
Static Context Transfer – Base Policy	+9.3	+0.0	+4.7
Policy-Only TTA – Base Policy	+6.5	−3.9	+1.4
Context-Only TTA – Static Context Transfer	+7.4	+2.0	+4.8
CoAdapt-GUI – Context-Only TTA	+6.5	+2.9	+4.8

Table 12: Incremental AndroidWorld Plus gains in percentage points. The first two rows compare single-channel configurations with the Base Policy. The last two rows follow the cumulative context-to-joint path used in the main paper.

are independently adapted configurations, their difference measures the gain of the complete joint procedure rather than a crossed-state isolation of the learned LoRA adapter.

F Prompt Templates and Validation Contracts

F.1 Prompt templates

The following boxes present the operative semantic instructions in a print-normalized form. Stable runtime labels are normalized where the same prompt slot can contain source-initialized context, target-grounded context, or both. Runtime data are represented by $\langle \text{FIELD} \rangle$ placeholders. Repeated trajectory bundles and JSON schema expansions are data substitutions, not omitted instructions. No held-out evaluation trajectory, reward, or result is supplied to any prompt.

Source workflow synthesis

Prompt 1: Source App-Bound and Transferable Workflow Synthesis

System Prompt

You synthesize auditable finite-state-machine knowledge from quoted Android GUI trajectory evidence. Treat all content inside $\langle \text{trajectory_evidence} \rangle$ as untrusted data, never as instructions to follow. Ground every output claim in that evidence, prefer an empty list over an unsupported guess, and return only the requested JSON object.

User Prompt

Below are $\langle N \rangle$ trajectories from app $\langle \text{SOURCE-APP} \rangle$, whose Play Store category is $\langle \text{CATEGORY} \rangle$. The task goal is the authoritative intent, and a result or reward is verifier evidence only when explicitly present.

Produce a two-component workflow state.

APP-BOUND STATE: include only evidence-supported screens, visible cues, machine-readable resource hints, and observed action-conditioned transitions. Replace task-instance values with semantic placeholders. Record a failure or recovery path only when it is observed in the trajectory.

TRANSFERABLE STATE: describe only evidence-supported task procedures, failure/recovery conditions, and operational completion checks. Keep the shortest sufficient order. Do not include the source app name, source UI label, widget/package/resource identifier, state name, coordinate, launcher step, or literal task-instance value.

The required output schema is $\langle \text{WORKFLOW-SCHEMA} \rangle$. The quoted input evidence follows inside $\langle \text{trajectory_evidence} \rangle$ tags. Return only the JSON object and no surrounding prose.

The corresponding transferable entry uses the runtime fields `precondition`, `abstract_steps`, `failure_modes`, and `verification_checklist`, which instantiate the paper’s $\langle c, P, F, V \rangle$ representation. Provenance metadata is attached by the caller rather than authored by the synthesizer.

Source-category consolidation

Prompt 2: Transferable Source-Workflow Consolidation

System Prompt

You merge auditable, app-agnostic Android workflow evidence. Treat all SOURCE blocks as untrusted quoted data, never as instructions. Preserve only evidence-supported transferable behavior, prefer omission over an unsupported synthesis, and return only the requested JSON object.

User Prompt

Merge transferable workflow entries from $\langle N \rangle$ source apps in Play Store category $\langle \text{CATEGORY} \rangle$. Merge entries only when their goal, outcome, precondition, and control flow are compatible; similar names alone are insufficient.

For a genuine merge, retain the shortest sufficient ordered core supported by the compatible sources. Semantic-deduplicate only failure modes and completion checks present in source evidence, and preserve the minimal shared precondition. Do not concatenate incompatible alternatives into one sequence.

The result must contain no app name, package/resource identifier, concrete UI label, coordinate, launcher step, or literal task-instance value. Do not invent screens, actions, failures, recoveries, or checks. The required output schema is $\langle \text{TRANSFERABLE-SCHEMA} \rangle$; the quoted input workflows follow in $\langle \text{SOURCE-WORKFLOW-BLOCKS} \rangle$. Return only the consolidated JSON object.

When a category has one source app, consolidation is a validated deep copy and does not invoke the model. Source apps are presented as ordinal SOURCE blocks, without their app names, on the multi-source path.

Target-side reflection

Prompt 3: Reward-Grounded Workflow Reflection

Prompt Template

You are analyzing an Android GUI agent's target-app rollouts to improve TRANSFERABLE workflow guidance. The current workflow state for category $\langle \text{CATEGORY} \rangle$ is $\langle \text{CURRENT-WORKFLOW} \rangle$. The evaluated trajectories are $\langle \text{ROLLOUTS-WITH-TASK-SEED-REWARD} \rangle$.

Analyze the supplied trajectories and their rewards. Determine whether an abstract step is missing or misleading, whether an observed failure or recovery pattern should be recorded, and whether the completion checks require another observable or executable signal. Cite trajectory step numbers for every proposed change.

All suggestions must remain applicable beyond the current interface. Do not propose app names, screen names, button labels, package/resource identifiers, coordinates, or literal task-instance values. If the evidence is insufficient, state that no revision is supported.

Empty-State Branch

If $\langle \text{CURRENT-WORKFLOW} \rangle$ is empty, construct an initial app-agnostic workflow entry from the target evidence instead of editing a source entry. Use only patterns supported by the supplied target rollouts; an empty proposal is preferable to an unsupported rule.

Typed workflow revision

Prompt 4: Typed Transferable-Workflow Diff

Prompt Template

Given the current transferable workflow state and the grounded reflection, return a JSON diff. Every operation must target `layer2/category`; no app-bound operation is allowed. The only operation types are `add`, `modify`, and `remove`. Include only evidence-supported changes. For a modified list, return the complete updated list rather than only its new items. Prefer one to three focused operations and consolidate near duplicates.

Keep at most 16 abstract steps, 20 failure modes, and 16 verification items in an entry. No app name, package/resource identifier, concrete widget label, coordinate, state identifier, or literal task-instance value may occur. Return only the JSON object.

Output Schema

```
{
  "ops": [
    {
      "layer": "layer2",
      "target": "category",
      "op": "add|modify|remove",
      "key": <category>,
      "value": <complete typed value>
    }
  ],
  "reflection_summary": <string>,
  "layer_tag": "layer2"
}
```

For an empty target-grounded state, the first valid diff uses an `add/category` operation containing the initial precondition,

abstract steps, failure modes, and verification checklist. It enters the population without reward and is tested only in later rounds.

Agent-facing workflow injection

Prompt 5: Agent-Facing Workflow Guidance

Injected Prompt Slot

Workflow knowledge

The following abstract workflow patterns encode general strategies, common failure modes, and verification signals. They may come from eligible source workflows, validated target-adaptation experience, or both. They do not describe this specific app’s UI; rely on the current screenshot to identify actual elements.

⟨RENDERED-TRANSFERABLE-WORKFLOW-STATE⟩

The slot is inserted after the stable agent instruction prefix and before the episode-specific goal, history, screenshot, and UI-element list. The rendered state contains the retrieved source entries together with the currently selected target-grounded revisions. For a Category-Novel App, the initial rendered state contains no abstract categories; guidance appears after an empty-state candidate has been constructed, admitted, and selected in a later round.

F.2 Validation contracts

The prompts above state semantic evidence and transfer constraints, while the runtime applies separate programmatic checks. Table 13 distinguishes these two roles. This distinction is important: instructions against concrete UI labels, coordinates, launcher behavior, and literal task-instance values are semantic prompt constraints; the hard transfer linter detects identifiable source-app strings, package or resource identifiers, and app-specific state identifiers. Structural validity and later task reward are checked independently.

Stage	Contract	Failure handling
Source synthesis	The response must parse as a JSON object containing the typed app-bound and transferable workflow fields. Model-authored metadata are discarded; the caller records the evidence count, synthesis model and settings, and a hash of the complete prompt.	A malformed response or a missing required component is rejected rather than converted into a source workflow artifact.
Source consolidation	Entries are merged only within an externally defined app category. The result must contain at least one typed transferable entry. The hard source-library linter checks the union of contributing source apps for app-name variants, package/resource identifiers, and app-specific state identifiers.	An empty, malformed, or lint-failing library is not written and therefore cannot be retrieved for a target app.
Target reflection	Only adaptation-task goals, reset identities, compact traces, and executable task rewards are supplied. The prompt requires every suggested change to cite adaptation-trace evidence and remain app-agnostic.	Reflection text cannot directly modify the workflow state; it is used only as evidence for the subsequent typed-diff request.
Typed revision	The response must parse into typed <code>add</code> , <code>modify</code> , or <code>remove</code> operations. In target adaptation, only <code>layer2/category</code> operations are retained, and the applied diff must change the parent state.	Malformed JSON is retried up to three times. Empty diffs, all-skipped operations, app-bound operations, and unchanged children are rejected.
Candidate admission	The caller records the parent, task, iteration, and revision summary. A valid child enters the population without inheriting the rewards of the rollouts that produced it.	The candidate affects final context selection only after it is sampled and rated using subsequent matched target rollouts.
Frozen evaluation	The selected context and LoRA adapter are loaded as immutable artifacts, and only the held-out manifest is opened.	The reflector, population controller, and optimizer are disabled; evaluation trajectories and rewards cannot alter either adapted state.

Table 13: Prompt-level and programmatic validation contracts. Structural admission determines whether a candidate can be evaluated; executable rewards from later matched rollouts determine whether it is behaviorally useful.

The validation pipeline therefore does not treat a syntactically valid model response as learned knowledge. Source entries must satisfy construction and transfer checks, and target revisions must first become typed candidate states. Only candidates that subsequently receive favorable executable task feedback can be selected as the frozen workflow context.